

Practical 4 :

Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?

- Kill processes by name
- Kill a process based on the process name
- Kill a single process at a time with the given process ID

```
HP@DESKTOP-B4BD73N MINGW64 ~  
$ sleep 500 &  
[1] 116  
  
HP@DESKTOP-B4BD73N MINGW64 ~  
$ kill 116
```

```
HP@DESKTOP-B4BD73N MINGW64 ~  
$ sleep 500 &  
[2] 117  
[1] Terminated sleep 500  
  
HP@DESKTOP-B4BD73N MINGW64 ~  
$
```

2. Write a program for process creation using C

- Orphan Process

```
GNU nano 8.7
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(5);
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
    }
    else {
        printf("Parent exiting\n");
    }
    return 0;
}
```

```
HP@DESKTOP-B4BD73N MSYS ~
$ nano orphan.c

HP@DESKTOP-B4BD73N MSYS ~
$ gcc orphan.c -o orphan

HP@DESKTOP-B4BD73N MSYS ~
$ ./orphan
Parent exiting

HP@DESKTOP-B4BD73N MSYS ~
$ Child Process
PID : 482
PPID : 1
```

- Zombie Process

```
M ~
GNU nano 8.7
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child exiting\n");
    }
    else {
        sleep(10);
        printf("Parent running\n");
    }
    return 0;
}
```

```
HP@DESKTOP-B4BD73N MSYS ~
$ nano zombie.c

HP@DESKTOP-B4BD73N MSYS ~
$ gcc zombie.c -o zombie

HP@DESKTOP-B4BD73N MSYS ~
$ ./zombie
Child exiting
Parent running

HP@DESKTOP-B4BD73N MSYS ~
$
```

3. Create the process using fork () system call.

- Child Process creation
- Parent process creation
- PPID and PID

```
HP@DESKTOP-B4BD73N MSYS ~
$ gcc fork.c -o fork
cc1: fatal error: fork.c: No such file or directory
compilation terminated.

HP@DESKTOP-B4BD73N MSYS ~
$ ./fork
-bash: ./fork: No such file or directory

HP@DESKTOP-B4BD73N MSYS ~
$ nano fork.c

HP@DESKTOP-B4BD73N MSYS ~
$ gcc fork.c -o fork
```

```
~
GNU nano 8.7                                     fork.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
    }
    else {
        // Parent process
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
        wait(NULL);
    }
    return 0;
}
```